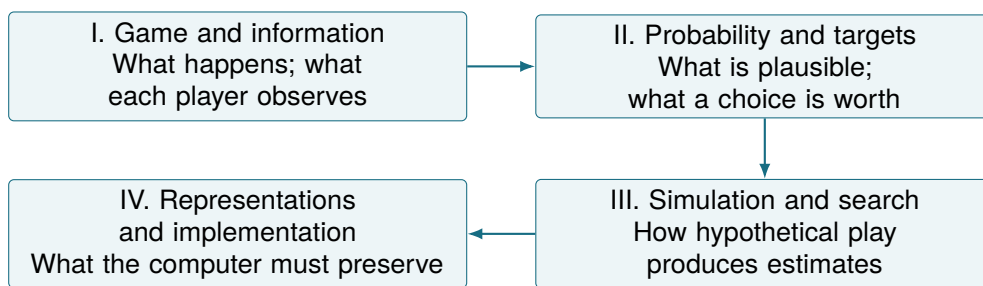


Information, Belief, and Search

A formal reference and guided introduction
to imperfect-information game search



Purpose. A reference and a teaching document: definitions, invariants, short proofs, worked examples, and the conditions that connect theoretical objects to a practical game-search implementation.

Scope. Two players; deterministic engine advancement; potentially randomized player choices; imperfect information; full observation histories.

Version 1.4 / 9 September 2026

A guided development with worked examples and a formal reference.

Abstract

A player sees a sequence of observations and must choose an action without knowing the complete game history. A signal game lets us follow that decision: observations narrow the possibilities, a model of the sender weights them, and a rule for future choices determines expected payoffs. Worked updates and a search trace show how those probabilities become action estimates. We also show why a search can use only the root player’s information yet give its simulated opponent an information advantage: two sender-root searches can produce different guesses from the same opponent observations. The final part asks what an implementation must retain to reproduce these decisions and values, and connects the objects to MTGallium source. Definitions, proofs, exercises, and separate solutions provide a reference alongside the examples.

Contents

1	Begin with a private signal	1
1.1	A reading map	1
1.2	Three kinds of statement	1
I	Game and information	2
2	Game states, choices, and advancement	2
2.1	Determinism and hidden information	2
2.2	Histories and reconstruction	3
3	Observations and player history	3
3.1	Recovering one’s own choices	4
3.2	Recovering decision status and legal choices	5
4	Compatibility and epistemic state	5
4.1	Three different compatible sets	6
4.2	Remembering facts without assigning probabilities	7
4.3	Why the full epistemic state is sufficient for continuation	7
II	Probability and decision targets	8
5	Policies and probabilities of play	8
5.1	Path probability	8
6	Weighting compatible histories	8
6.1	The natural endpoint: encountering a decision	9
6.2	Other history endpoints	9
6.3	Derived beliefs preserve joint relationships	10
6.4	Own choices versus observed opponent choices	10
6.5	Advancing and updating a belief	10
7	Payoff and target values	11
7.1	Belief value and action value	11
7.2	Defining the value target	12
8	Evaluators and representations of their inputs	12
8.1	Range and target identity	13
III	Simulation and search	13
9	Hypothetical simulation	13
9.1	Hypothesis consistency and failure	14

9.2	Cutoffs and settlement	14
10	Search nodes, action branches, and statistics	14
10.1	Search-local statistics	15
10.2	Watching the guesser search	15
10.3	How a node acquires a population	16
11	A complete search iteration	16
11.1	Selection from the current search record	17
11.2	Which opponent is being simulated?	18
11.3	Expansion and action coverage	19
11.4	Backup and root choice	19
11.5	Evaluating a cutoff at the opponent's turn	19
12	Connecting search averages to a target	20
12.1	A fixed-action reference experiment	20
12.2	Visitation changes the population	20
12.3	Adaptive continuation, stopping, and execution	21
12.4	Budgets, reuse, and uncertainty	21
IV	Representations and implementation	22
13	Representation contracts	22
13.1	Perspective safety as factorization	22
13.2	The representation map	22
13.3	Completeness and task sufficiency	23
13.4	Execution fidelity	23
13.5	Incremental history and knowledge summaries	23
13.6	Simulation and search-state isolation	24
13.7	Composing a perspective-safe root policy	24
14	Symmetry between players	25
14.1	Exchanging policies	26
14.2	Exchanging beliefs, values, and evaluators	26
15	Where these objects appear in MTGallium	27
15.1	Application boundary of the deterministic model	27
16	Connections to the literature	28
17	Scope boundaries and a specification template	28
A	Notation reference	29
B	Compact contract sheet	30
C	Exercises	31
D	Worked solutions	33
	References	35

1 Begin with a private signal

A sender knows a private type, **orange** or **blue**. The sender chooses a public signal, and a second player tries to guess the type. The same signal can come from either type. What should the guesser do, and how could a search program decide?

The game is small, but answering those questions calls for the same objects that a card-game planner uses. A game history tells us which type the sender actually has and which signal was chosen. The guesser observes only part of that history. A belief assigns probabilities to the remaining possibilities. Those probabilities, together with a rule for future choices, determine expected payoffs. Search estimates the payoffs by trying hypothetical continuations.

We will return to this game as each object becomes available. Sender is player 0, guesser is player 1, and the hidden type is $\theta \in \{\text{O}, \text{B}\}$. The signals are ℓ (left) and r (right); the guesses are g_{O} and g_{B} . A correct guess wins for the guesser; an incorrect guess wins for the sender. The type is fixed before play. Only player choices may randomize.

The question that guides the construction

After seeing ℓ , the guesser must choose the same distribution over guesses in the orange and blue cases whenever its observations are identical. Search can examine both cases to estimate their consequences. The challenge is to use those hypotheses to choose well with the information the player actually has.

1.1 A reading map

Part I builds game histories and projects them into each player's experience. It then collects the histories compatible with that experience. **Part II** adds policies and probabilities, leading to a concrete calculation of what the two guesses are worth. **Part III** follows search iterations and their statistics, including the different ways to model a simulated opponent. **Part IV** gives conditions under which the entire computation preserves the root player's information boundary and connects the objects to MTGallium's representations.

[Sections A](#) and [B](#) collect notation and classified contracts; [sections C](#) and [D](#) provide exercises and solutions.

This paper brings together standard ideas from extensive-form games, partially observable models, and Monte Carlo search to specify the information and value semantics of MTGallium. [Section 16](#) gives the correspondence to the literature; [section 15](#) maps the objects to source.

1.2 Three kinds of statement

Model assumptions fix the game we study: for example, automatic processing terminates. *Mathematical consequences* follow from those assumptions and definitions: for example, indistinguishable decision histories have the same legal menu. *Implementation obligations and algorithm choices* explain how a program realizes the objects and allocates computation. This classification will also organize the reference sheet.

We use finite action sets and finite or countable state and observation spaces, so probabilities can be written as sums.

Part I

Game and information

2 Game states, choices, and advancement

A *game state* is a state awaiting a player choice or a terminal state. Advancement collects the automatic rule processing between these points into one transition. An engine can retain finer intermediate configurations for execution and diagnosis.

Definition 2.1 (Deterministic game skeleton). *A game skeleton consists of*

$$\mathcal{G}_{\text{dyn}} = (P, S, S_{\text{init}}, F, A, p, A, \text{next}),$$

where $P = \{0, 1\}$ is the player set, S is the game-state space, $\emptyset \neq S_{\text{init}} \subseteq S$ is the allowed initial-state set, $F \subseteq S$ is the terminal-state set, and A is an action universe. For every nonterminal state s ,

$$p(s) \in P, \quad \emptyset \neq A(s) \subseteq A,$$

with $A(s)$ finite. Advancement is a function

$$\text{next} : \{(s, a) : s \notin F, a \in A(s)\} \rightarrow S.$$

The action a includes the semantic content of the choice. If a target or an ordering requires another player choice, advancement stops at that decision. If a game presents an entire compound choice at once, the action can encode that complete choice. Granularity follows the actual decision interface rather than an arbitrary size of engine instruction.

Contract. For each legal (s, a) , advancement consumes that action and all automatic consequences, then reaches the next player choice or termination after finitely many automatic steps.

2.1 Determinism and hidden information

Given the same s and a , the same next state results. The acting player need not alternate: $p(\text{next}(s, a))$ can equal $p(s)$. Terminal states have neither an acting player nor an outgoing action transition. We leave p, A undefined there.

Drawing the next card from a fixed hidden library order is deterministic: the order in s determines the card, even when the player is uncertain about it. A belief or prior will describe that uncertainty. Fresh random shuffles require a chance-transition extension, outside the present model. Here randomness enters through player policies and through the distribution used to describe possible initial states.

Example 2.2 (Automatic consequence versus additional choice). A player chooses to cast a spell. Paying a fully specified cost and putting the spell on the stack may be automatic consequences of that action. Asking the other player whether to respond is a new game state. Resolving an effect that asks someone to choose a card likewise reaches a new game state. These decision points determine the granularity of the history.

2.2 Histories and reconstruction

Definition 2.3 (Game history). *A valid finite game history is*

$$h_n = (s_0, a_0, s_1, \dots, a_{n-1}, s_n),$$

where $s_0 \in S_{\text{init}}$, and for $0 \leq j < n$, $s_j \notin F$, $a_j \in A(s_j)$, and $s_{j+1} = \text{next}(s_j, a_j)$. Write $\mathcal{H}$ for all such histories, $|h_n| = n$, and $\text{last}(h_n) = s_n$.

Write $h \leq h'$ when h is a prefix of h' . Given s_0 and a legal action sequence, the game history is uniquely reconstructible by repeated application of next . This reconstructs all game states at our chosen granularity. It does not select a unique low-level engine trace unless an implementation separately specifies that trace.

Proposition 2.4 (History reconstruction). *Two valid histories with the same initial state and the same action sequence have the same state sequence.*

Proof. Their first states agree. If their states at index j agree, then the common action a_j and the function next give the same state at index $j + 1$. Induction completes the argument. $\square$

An unbounded sequence of individually terminating choices remains possible. [Section 7](#) supplies the additional termination assumption needed for terminal expected values.

The signal game

Initial states s^θ await the sender's choice:

$$p(s^\theta) = 0, \quad A(s^\theta) = \{\ell, r\}.$$

After a signal m , state $t^{\theta, m}$ awaits a guess:

$$\text{next}(s^\theta, m) = t^{\theta, m}, \quad p(t^{\theta, m}) = 1, \quad A(t^{\theta, m}) = \{g_{\text{O}}, g_{\text{B}}\}.$$

A guess advances to terminal state $f^{\theta, m, g}$. Thus each complete play has two transitions, one for the signal and one for the guess.

3 Observations and player history

Observation rules specify who learns what during play. A private reveal can disclose a card identity without changing the eventual game configuration. Conversely, an opponent can change a hidden configuration without disclosing its new contents. Observation rules are additional components of the game.

Definition 3.1 (Observation functions). *For each player $q \in P$, let Ω_q be an observation alphabet and Ω_q^* its finite words, including the empty word ε . Specify*

$$O_q^0 : S_{\text{init}} \rightarrow \Omega_q^*, \quad O_q : \text{dom}(\text{next}) \rightarrow \Omega_q^*.$$

$O_q^0(s_0)$ is the initial observation sequence. $O_q(s, a)$ is the ordered sequence disclosed during advancement by a , including any resulting request for a choice. Concatenation is written $\parallel$.

One transition may contribute several observations. An empty contribution leaves the observation word unchanged, preserving the privacy of an undisclosed transition. Timing, turn, and phase information enter this word precisely when the game makes them available to the player.

Contract. The chosen game state contains all context needed to determine both advancement and its disclosures. In particular, equal (s, a) pairs produce equal $O_q(s, a)$ for each player.

This assumption makes the observation function usable from a current configuration. For example, a permission to inspect a card belongs in that configuration when it affects a forthcoming disclosure. A history-dependent disclosure rule requires either retaining the relevant context in the state or giving the observation function a history argument. We use the former formulation throughout.

Definition 3.2 (Player history and information state). For $h_n \in \mathcal{H}$, define

$$I_q(h_n) = O_q^0(s_0) \parallel O_q(s_0, a_0) \parallel \cdots \parallel O_q(s_{n-1}, a_{n-1}). \quad (1)$$

The player history and the theoretical information state are this same object. Let $\mathcal{I}_q = \{I_q(h) : h \in \mathcal{H}\}$ be the realizable information states. A current snapshot can summarize part of this accumulated experience.

Example 3.3 (Disclosed information and remembered consequence). Earlier, a player observed “the top library card is Lightning Bolt.” Later, they observe “the opponent draws that top card.” The second observation need not repeat the card identity. Together the observations entail a remembered fact about the opponent’s hand, provided no intervening event invalidated it. The observation function supplies disclosures; reasoning over the history supplies their consequences.

The signal game

For readability, write $\text{request}(M)$ for a request to choose from menu M , $\text{chose}(a)$ for one’s own realized choice, and $\text{signal}(m)$ for an observed public signal. These are typed observation tokens, not engine identifiers.

The sender initially receives its type and a signal request. The guesser initially receives its role. On the signal transition, the guesser receives the public signal and a request to guess. In particular,

$$\begin{aligned} I_0(s^\theta) &= (\text{type}(\theta), \text{request}(\{\ell, r\})), \\ I_1(s^\theta, \ell, t^{\theta, \ell}) &= (\text{role}(\text{guesser}), \text{signal}(\ell), \text{request}(\{g_{\text{O}}, g_{\text{B}}\})) =: i_\ell. \end{aligned}$$

The second line is identical for $\theta = \text{O}$ and $\theta = \text{B}$. On a guess transition the guesser receives its own-choice token and the outcome. The sender likewise receives the outcome after its earlier own-signal token. These records explain both who must act and what that player remembers.

3.1 Recovering one’s own choices

Define the action subsequence

$$\text{own}_q(h_n) = (a_j : 0 \leq j < n, p(s_j) = q).$$

Our own-choice requirement is the existence of

$$R_q : \mathcal{I}_q \rightarrow \mathcal{A}^*, \quad R_q(I_q(h)) = \text{own}_q(h). \quad (2)$$

Equivalently, equal player histories imply equal own-action subsequences. This applies to every valid prefix, not just completed games.

Perfect recall includes what the player knew when choosing each action. Place each own-choice token before the observations generated by that choice: when $p(s) = q$, let $O_q(s, a)$ begin with the typed token chose_q(a), followed by the resulting disclosures. Initial and unrelated observations use other token types. The prefix before each own-choice token then records exactly the information available when the action was selected, so one can recover

$$((I_q(h_j), a_j) : p(s_j) = q),$$

together with the order of observations between choices. Other encodings can preserve the same chronology.

3.2 Recovering decision status and legal choices

For every state, define

$$d_q(s) = \begin{cases} A(s), & s \notin F \text{ and } p(s) = q, \\ \perp, & \text{otherwise.} \end{cases}$$

The symbol $\perp$ means that no choice is currently required from q . It does not distinguish an opponent decision from termination. If terminal status is itself disclosed, that is supplied by the observation rules.

Require a decoder

$$D_q : \mathcal{I}_q \rightarrow \mathcal{D}_{\text{fin}}(A) \cup \{\perp\}, \quad D_q(I_q(h)) = d_q(\text{last}(h)). \quad (3)$$

Only nonempty finite sets occur as decision values. A concrete implementation need not print the full menu, but the theoretical history must determine it. Typed request and action tokens are one sufficient encoding.

Proposition 3.4 (Decision consistency). *If $I_q(h) = I_q(h')$, then $d_q(\text{last}(h)) = d_q(\text{last}(h'))$. In particular, if one is a decision for q , both are, and their legal action sets are equal.*

Proof. Apply the same function D_q to the common information state. □

When two situations have different legal menus, check whether the player can observe the difference. An observable menu difference belongs in the information state. If the player receives no distinguishing observation, either the action abstraction or the observation model needs revision.

4 Compatibility and epistemic state

Definition 4.1 (Compatible histories). *For realizable $i_q \in \mathcal{I}_q$, define*

$$C_q(i_q) = \{h \in \mathcal{H} : I_q(h) = i_q\}.$$

These are the possibilities allowed by the game and observation rules.

Define $h \sim_q h'$ exactly when $I_q(h) = I_q(h')$. Equality makes this relation reflexive, symmetric, and transitive. Its equivalence classes are the sets $C_q(i_q)$; two such sets are identical or disjoint. The actual history is always included:

$$h \in C_q(I_q(h)).$$

Own-choice and decision consistency hold throughout each class.

4.1 Three different compatible sets

Define the full epistemic state of a history by

$$E(h) = (\text{last}(h), I_0(h), I_1(h)). \quad (4)$$

“Epistemic” means concerning information or knowledge. The full tuple records the current configuration together with both players’ experiences. Its continuation role is established below. The compatible current-state sets are

$$\mathcal{S}_q(i_q) = \{\text{last}(h) : h \in C_q(i_q)\}, \quad (5)$$

$$\mathcal{E}_q(i_q) = \{E(h) : h \in C_q(i_q)\}. \quad (6)$$

The player information-state component i_q is fixed in the second set; the other player’s component need not be. Both sets are images of a compatibility class: applying each projection gathers its possible current tuples or states.

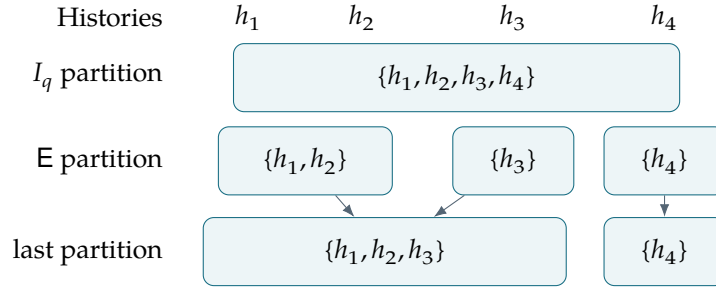


Figure 1: An illustrative family with one fixed I_q . Histories h_1, h_2 differ only in discarded past detail and share the full tuple. History h_3 has the same current game state but different opponent memory; h_4 has a different current state. Thus this family yields three epistemic tuples and two game states. Globally, game-state and player-information partitions can cross; the full tuple retains both.

Example 4.2 (The same game state, different remembered information). Two histories end with the same cards in the same zones, the same resources, and the same player to act. In one, the opponent earlier observed a particular card in the player’s hand; in the other, they did not. If the earlier reveal leaves no current rule-relevant flag, the final game states can agree while the opponent’s information histories differ. The full epistemic states remain distinct even though their first components coincide.

Compatibility is joint. A game state from $\mathcal{S}_q(i_q)$ and an opponent information state occurring somewhere in $\mathcal{E}_q(i_q)$ cannot necessarily be combined. A valid tuple needs a common witnessing history. Independently sampling these components can fabricate a situation that never occurred in any compatible play.

The guesser after the left signal

After the left signal, the guesser has information state i_ℓ . Exactly two histories in the basic game produce it:

$$C_1(i_\ell) = \{(s^\circ, \ell, t^{\circ, \ell}), (s^B, \ell, t^{B, \ell})\}.$$

Their current game states differ in private type. Their full epistemic tuples also differ because the sender remembers that type. The guesser’s component is the same. A single guess therefore has two possible consequences.

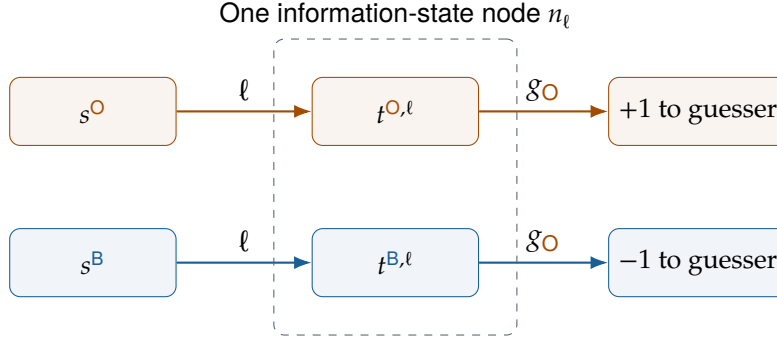


Figure 2: Two concrete trajectories share a guesser decision node. Choosing g^O has different outcomes in its two compatible hypotheses. Only the ℓ branch is shown.

4.2 Remembering facts without assigning probabilities

For a predicate φ on game histories, say it is entailed by i_q when

$$\forall h \in C_q(i_q), \quad \varphi(h).$$

For a current-game-state predicate ψ , use $\forall s \in \delta_q(i_q), \psi(s)$. These are compatibility-based facts. A model giving φ probability 0.99, or even probability 1 after assigning zero mass to compatible exceptions, is a different claim.

As observations accumulate, player histories extend by concatenation. The compatible sets of complete histories at two different moments are not generally nested: their members refer to different endpoints. Likewise, sets of compatible current game states need not shrink while the game changes. Each transition extends the actual history and appends its disclosed observations. To compare possibilities before and after that transition, align their time or decision endpoints; see [section 6](#).

4.3 Why the full epistemic state is sufficient for continuation

Although two histories can share $E(h)$, their discarded details do not affect future behavior *under this model*. Define

$$T((s, i_0, i_1), a) = (\text{next}(s, a), i_0 \parallel O_0(s, a), i_1 \parallel O_1(s, a)). \quad (7)$$

Proposition 4.3 (Continuation sufficiency). *Fix policies whose action distributions depend only on player information states. Histories with the same full epistemic state induce the same law of future full epistemic states and, when defined, terminal payoff.*

Proof. The common game state gives the same actor and legal actions. The actor's common information state gives the same action distribution. For each realized action, [equation \(7\)](#) gives the same successor tuple. Induct on the number of future transitions. Terminal payoff depends on the terminal game state, so its law agrees as well. $\square$

The tuple therefore provides a lossless continuation representation for this fixed-policy model. An implementation can use this result by retaining all state context that affects transitions and disclosures, together with each player's observation history. Histories remain a convenient foundation for defining how those components arise.

Part II

Probability and decision targets

5 Policies and probabilities of play

The observation history tells us which actions a player can choose. To describe how play proceeds, we also need a rule for choosing among them. After ℓ , one guesser policy always chooses orange; another chooses orange with probability 3/4 and blue with probability 1/4.

Write $\mathcal{I}_q^{\text{act}} = \{i_q : D_q(i_q) \neq \perp\}$ and, on that set, $A_q(i_q) = D_q(i_q)$. For a finite or countable set X , $\Delta(X)$ is the set of probability distributions on X : nonnegative masses summing to one. Action sets themselves remain finite.

Definition 5.1 (Behavioral policy). *A behavioral policy for player q assigns a distribution over legal choices:*

$$\pi_q(\cdot \mid i_q) \in \Delta(A_q(i_q)) \quad \text{for every } i_q \in \mathcal{I}_q^{\text{act}}.$$

A pair $\pi = (\pi_0, \pi_1)$ is a *policy profile*. At each decision, the acting player's policy samples an action and deterministic advancement gives the next state. The two guesser policies above therefore describe different patterns of play, but each uses the same choice probabilities under orange and blue whenever the guesser's observations agree.

Randomization. Conditional on the full history, the next action has the law $\pi_q(\cdot \mid I_q(h))$. Independent fresh draws realize this law; persistent random memory or shared randomness needs a richer model if it changes that conditional distribution.

5.1 Path probability

Fix an initial-state distribution $\lambda \in \Delta(S_{\text{init}})$. A point mass describes a fixed starting state. For a valid history h_n , its prefix probability under λ and π is

$$\mathbb{P}_{\lambda, \pi}([h_n]) = \lambda(s_0) \prod_{j=0}^{n-1} \pi_{p(s_j)}(a_j \mid I_{p(s_j)}(h_j)). \quad (8)$$

The notation $[h_n]$ means the event that play begins with this prefix. There is no chance-transition factor. Histories at a fixed depth are disjoint prefix events, but a history and its extension are not: $[h'] \subseteq [h]$ when $h \leq h'$. Therefore [equation \(8\)](#) is not a normalized mass function over *all* finite histories taken together.

6 Weighting compatible histories

Definition 6.1 (Belief at a player information state). *A belief at i_q is a probability distribution*

$$b_q(\cdot \mid i_q) \in \Delta(C_q(i_q)).$$

It assigns zero mass outside compatibility. It need not assign positive mass to every compatible history.

A prior and a model of past play provide one way to choose the weights. In the signal game, we ask how frequently each type of sender would choose ℓ .

Example 6.2 (A signal changes probability without eliminating possibility). A sender privately knows a type O or B . The observer's initial prior is $1/2$ each. Both types can legally choose public signal ℓ . Under a particular sender policy,

$$\mathbb{P}(\ell \mid \text{O}) = 4/5, \quad \mathbb{P}(\ell \mid \text{B}) = 1/5.$$

After ℓ , both types remain compatible, but

$$\mathbb{P}(\text{O} \mid \ell) = \frac{(1/2)(4/5)}{(1/2)(4/5) + (1/2)(1/5)} = 4/5.$$

From possibility to probability

Observing ℓ leaves both orange and blue compatible. A sender model that chooses ℓ more often when orange makes orange more probable. The observation determines which histories fit; the model determines their relative weights.

6.1 The natural endpoint: encountering a decision

Lemma 6.3 (Decision-state compatibility is prefix-free). *For $i_q \in \mathcal{I}_q^{\text{act}}$, no member of $C_q(i_q)$ is a proper prefix of another member.*

Proof. Suppose $h < h'$ are both members. Decision consistency says that q must act at the end of h . The extension h' therefore contains an additional action of q , so $\text{own}_q(h')$ is strictly longer than $\text{own}_q(h)$. Own-choice recoverability says that equal information states have equal own-action sequences, giving a contradiction. $\square$

Thus a play can encounter a particular acting information state at most once. Its compatible prefix events are disjoint. Let

$$z_q(i_q) = \sum_{h \in C_q(i_q)} \mathbb{P}_{\lambda, \pi}([h]).$$

This is the probability that play encounters that decision information state. When it is positive, the reach-conditioned posterior is

$$b_q^{\text{reach}}(h \mid i_q) = \frac{\mathbb{P}_{\lambda, \pi}([h])}{z_q(i_q)}, \quad h \in C_q(i_q). \quad (9)$$

The denominator is at most one, even if compatible histories have different lengths. This is the usual planner-root case: condition on arriving at the decision the player is actually facing.

6.2 Other history endpoints

To derive a belief by conditioning generated play, first specify a random finite history H . For example, stop at player q 's k th decision, provided that decision occurs; the experiment can explicitly condition on its occurrence. Alternatively choose a declared prefix-free frontier: a set of histories no one of which is a proper prefix of another. Restrict and normalize the path probabilities on that frontier when its total probability is positive.

Then, when $\mathbb{P}(I_q(H) = i_q) > 0$,

$$b_q(h \mid i_q) = \frac{\mathbb{P}(H = h) \mathbf{1}\{I_q(h) = i_q\}}{\sum_{g: I_q(g) = i_q} \mathbb{P}(H = g)}. \quad (10)$$

The law of H includes the prior, policy profile used to generate the past, and endpoint selection rule. [Lemma 6.3](#) supplies a canonical reach event for acting information states. For a waiting information state, unobserved transitions can leave I_q unchanged, so compatible prefixes can overlap; the general experiment must specify which endpoint is sampled.

An endpoint is a modeling device, not necessarily a timestamp disclosed to the player. Conditioning on an undisclosed global step count would change their information if the model treated that count as known. A decision occurrence recoverable from the player's own action chronology is often a cleaner endpoint. At a probability-zero observation, ordinary conditioning does not specify a posterior; an explicit off-path model is needed.

6.3 Derived beliefs preserve joint relationships

Several compatible histories may end at the same current game state. To find that state's probability, add the probabilities of those histories. More generally, mapping histories through a function f produces a distribution called the *pushforward*, $f_{\#}b$:

$$(f_{\#}b)(x) = \sum_{h:f(h)=x} b(h).$$

Thus $\text{last}_{\#}b_q$ is a belief over current game states and $\mathbf{E}_{\#}b_q$ is a belief over full epistemic states. The latter keeps the correlation between game configuration and opponent information. Under [proposition 4.3](#), it is sufficient for continuation under the fixed information-state policies. The game-state marginal alone need not be.

6.4 Own choices versus observed opponent choices

At a decision information state i_q , the player's own action probability $\pi_q(a \mid i_q)$ is the same throughout $C_q(i_q)$. If it is positive, conditioning solely on the realization of that own action does not reweight those hypotheses: the common likelihood factor cancels. Subsequent disclosures can of course reweight them.

An opponent action is different. Compatible histories can give that opponent different information states and hence different action probabilities. The observed action can therefore change the player's posterior through the opponent model. This is inference from behavior, not an engine disclosure of the opponent's hidden information.

6.5 Advancing and updating a belief

Add a preliminary decision to the signal game: the guesser can request a signal or leave for a draw. Before choosing, it assigns mass 1/2 to each type. Fix its choice to *request*. That advances both hypotheses to the sender's decision, preserving their weights. The sender then chooses a signal with the policy from [example 6.2](#).

Type	Mass after request	Sender choice	Likelihood	Branch mass
O	1/2	ℓ	4/5	2/5
O	1/2	r	1/5	1/10
B	1/2	ℓ	1/5	1/10
B	1/2	r	4/5	2/5

Each branch contains an advanced game state and both updated observation histories. On observing ℓ and receiving the guess request, the guesser keeps the two ℓ branches. Their masses sum to $1/2$; normalizing gives $4/5$ for $(s^{\text{O}}, \ell, t^{\text{O}, \ell})$ and $1/5$ for $(s^{\text{B}}, \ell, t^{\text{B}, \ell})$, with the preliminary request prefix understood.

The update has three operations: advance each hypothesis under the chosen action, branch and weight by intervening opponent behavior, then match the new observation and normalize. The first operation changes the possible worlds; the latter two change how much probability each surviving world carries. The next guess uses this new distribution at the new decision state.

7 Payoff and target values

Definition 7.1 (Terminal payoff). For $q \in P$, let $u_q : F \rightarrow \{-1, 0, +1\}$ assign loss, draw, and win, respectively. We assume zero-sum payoff: $u_0(s) = -u_1(s)$.

The signal game's guesser receives $+1$ for a correct guess and -1 for an incorrect guess; the sender receives the opposite payoff. Expected payoff therefore measures the difference between win and loss probabilities.

Define $G_q^\pi(h)$ whenever continuation from h under π reaches a terminal state almost surely. It is the expected terminal payoff to q . At terminal h , it equals $u_q(\text{last}(h))$. At nonterminal h , write $s = \text{last}(h)$ and $r = p(s)$. Conditioning on the next action gives

$$G_q^\pi(h) = \sum_{\substack{a \in A(s) \\ \pi_r(a|I_r(h)) > 0}} \pi_r(a|I_r(h)) G_q^\pi(h \parallel (a, \text{next}(s, a))). \quad (11)$$

Positive-probability continuations inherit almost-sure termination. Restricting the sum to them keeps every continuation value in the expression defined. Bounded payoff makes this expectation finite even when expected game length is infinite. A model admitting nonterminating play needs an additional payoff convention for that play before assigning a terminal-value target to it.

7.1 Belief value and action value

The *support* of a belief contains its positive-mass histories: $\text{supp}(b_q) = \{h : b_q(h) > 0\}$. A compatible history can fall outside it. For example, if ℓ has positive likelihood under orange and zero likelihood under blue, blue remains rule-compatible after that signal but receives posterior mass zero. For a belief b_q at i_q , define

$$V_q(b_q, \pi) = \sum_{h \in \text{supp}(b_q)} b_q(h) G_q^\pi(h), \quad (12)$$

$$Q_q(b_q, a; \pi) = \sum_{h \in \text{supp}(b_q)} b_q(h) G_q^\pi(h \parallel (a, \text{next}(\text{last}(h), a))). \quad (13)$$

For V , continuation must terminate almost surely from every history in this support. For Q , $i_q \in \mathcal{I}_q^{\text{act}}$, $a \in A_q(i_q)$, and continuation after that forced first action must terminate almost surely on the support. Decision consistency makes a legal in every summand.

Q asks what happens when the player *chooses* a and then follows π . It remains meaningful when $\pi_q(a|i_q) = 0$: the first choice is an intervention, while later choices follow the policy.

The historical policy used to infer b_q can differ from the continuation policy being evaluated. This lets a player use a model of past behavior to compare new plans.

Proposition 7.2 (Value identities). *Let i_q be an acting information state and b_q a belief at i_q . Suppose continuation under π terminates almost surely on $\text{supp}(b_q)$, including after each forced legal action $a \in A_q(i_q)$. Then*

$$-1 \leq V_q(b_q, \pi) \leq 1, \quad -1 \leq Q_q(b_q, a; \pi) \leq 1, \quad (14)$$

$$V_q(b_q, \pi) = \sum_{a \in A_q(i_q)} \pi_q(a \mid i_q) Q_q(b_q, a; \pi). \quad (15)$$

Proof. The bounds follow by averaging payoffs in $[-1, 1]$. Substitute [equation \(11\)](#) into [equation \(12\)](#). Each next-action probability is constant on the compatibility class, so the finite action sum factors outside the history sum. Under the stronger forced-action assumption, the zero-weight action terms are also defined and can be included in [equation \(15\)](#). $\square$

Example 7.3 (Running game: choosing and mixing guesses). At i_ℓ , the posterior assigns probabilities $4/5$ to O and $1/5$ to B . Both guesses terminate immediately, so the action values are

$$\begin{aligned} Q_1(b_\ell, g_{\text{O}}; \pi) &= \frac{4}{5}(+1) + \frac{1}{5}(-1) = \frac{3}{5}, \\ Q_1(b_\ell, g_{\text{B}}; \pi) &= \frac{4}{5}(-1) + \frac{1}{5}(+1) = -\frac{3}{5}. \end{aligned}$$

If the guesser chooses g_{O} with probability $3/4$, its policy value is $V_1(b_\ell, \pi) = \frac{3}{4} \frac{3}{5} + \frac{1}{4}(-\frac{3}{5}) = \frac{3}{10}$. Thus $3/5$ is the value of choosing orange, while $3/10$ is the value of this randomized continuation policy at the same information state.

7.2 Defining the value target

To assign an expected payoff to an information state, fix a belief over compatible histories and policies for subsequent play. These choices specify the distribution of terminal outcomes. Writing the belief construction as $B_q : i_q \mapsto b_q$, the resulting target is $i_q \mapsto V_q(B_q(i_q), \pi)$. Changing B_q or π changes the question. A best response to a fixed opponent, a worst-case criterion, and an equilibrium objective each add their own optimization problem to this basic expectation.

Under one common distribution of terminal play, zero-sum payoffs give opposite expectations. Players with different observations may instead use different subjective distributions. Both can then assign themselves positive expected value; each expectation describes that player's model of the future.

8 Evaluators and representations of their inputs

Search can use an evaluator to score a position without continuing play to termination. We begin with a scorer that receives the player's observation history.

Definition 8.1 (Information-state evaluator). *An information-state evaluator for player q is a scoring function*

$$e_q : \mathcal{I}_q \rightarrow \mathbb{R},$$

or a function restricted to a specified set of information states, such as nonterminal decision contexts.

The project also calls this a *state evaluator*. The scoring function can be hand-designed, learned, or composite. To train one for the randomized policy in the running example, the exact target at i_ℓ is 3/10. To score the always-orange policy at the same input, the target is 3/5. The belief and continuation policy identify which of these tasks is intended.

A *sampled-world evaluator* has a game-state input, for example $g_q : S \rightarrow \mathbb{R}$ on a specified nonterminal domain. An evaluator on the full epistemic tuple can additionally use both players' represented histories. These richer inputs are useful inside trusted simulation; an information-state evaluator receives only the selected player's observation history.

Evaluating a hypothetical player's information

For a sampled history h , an information-state evaluator is called on $I_q(h)$. Projection extracts that hypothetical player's experience before scoring it. The scorer therefore uses the same input type in real and hypothetical play.

The implementation encodes this input before passing it to the scorer. [Section 13](#) develops the representation map and shows when a smaller encoding retains everything needed for a particular target.

8.1 Range and target identity

An evaluator's range is $\mathbb{R}$ unless restricted. A search that accepts only $[-1, 1]$ settlements can require this range or apply a declared transformation. Clipping and tanh produce new scoring functions; for example, rescaling a tactical heuristic into $[-1, 1]$ supplies a bounded search score, while estimating terminal payoff still requires a specified target and validation. [Section 11.5](#) works through a root-information evaluator called at an opponent's decision.

Part III

Simulation and search

9 Hypothetical simulation

At an actual root decision, fix root player q , information state i_q , and belief b_q . A simulation first samples $h \sim b_q$, choosing one possible history to explore under the root's model. In a computational approximation, a finite weighted collection of hypotheses, often called *particles*, supplies this sampling distribution. An exact full epistemic representation can replace the history for continuation under [proposition 4.3](#); a game-state sample alone is insufficient when opponent memory matters.

At a nonterminal simulated history, $r = p(\text{last}(h))$ acts using

$$a \sim \mu_r(\cdot \mid I_r(h)), \quad h \leftarrow h \parallel (a, \text{next}(\text{last}(h), a)).$$

This equation describes simulation under a genuine continuation policy profile. The sampled opponent acts on its own hypothetical observation history. Search will use the returns from these simulations to choose which actions to explore. One question will then arise: the simulated opponent acts on its own information, while the search record is built from the root's information. [Section 11.2](#) examines that relationship after we have seen the record and selector at work.

9.1 Hypothesis consistency and failure

Sample the game configuration and both player histories jointly so that one compatible history witnesses the tuple. After each action, both observation histories are updated with the same transition's respective observation sequences.

In the theoretical game, a legal action advances successfully. An adapter may fail to represent, bind, or execute it. Such a failure is an implementation outcome, not a new legal game outcome. Rejection, timeout, unsupported representation, and stopped execution must remain separately typed.

9.2 Cutoffs and settlement

A simulation can stop at terminal play, at a declared computational cutoff, or at an execution failure. Terminal play supplies $u_q(s)$. A nonterminal cutoff requires a declared scoring or continuation rule. A failure supplies no strategic value.

A *rollout* continues from a search leaf using declared action-selection policies. It uses the same game and observation rules. Its endpoint can be terminal, another cutoff, or failure. “Leaf” means that represented graph exploration stops for this iteration, not that the game has ended.

Record a successful settlement as (v, κ) , with origin κ identifying terminal payoff, a specific evaluator, or a specified bounded-rollout rule. The computation horizon, continuation policies, evaluator configuration, and payoff perspective contribute to its meaning. For example, a terminal +1 records a win in the hypothetical game; an evaluator score of +1 records that evaluator's judgment at its cutoff.

10 Search nodes, action branches, and statistics

A simulation follows a concrete hypothesis. A search structure determines which hypotheses contribute to the same decision statistics.

Definition 10.1 (Acting-player information-state node). *For a nonterminal history h , let*

$$K(h) = (p(\text{last}(h)), I_{p(\text{last}(h))}(h)).$$

A node $n = (r, i_r)$ groups all histories with the same acting player and the same information state for that player. This group is a fiber of K . Its legal action set is $A(n) = A_r(i_r)$.

Decision consistency makes the action set well-defined. Terminal histories produce outcome records rather than decision nodes. An action branch is the pair (n, a) , with $a \in A(n)$. It may have several successor decision nodes and terminal outcomes, since different histories in the fiber can produce different consequences for the same action.

10.1 Search-local statistics

The search record Z stores the actions explored and their returns. For each node-action pair, retain

$N(n, a)$ = number of value-bearing traversals,

$W(n, a)$ = sum of their backed root-perspective values.

When $N(n, a) > 0$, set $\widehat{Q}(n, a) = W(n, a)/N(n, a)$. For an unvisited action the mean is undefined; selection uses its declared exploration rule. Attempt, failure, proposal, and origin counts describe the additional computation that produced these value-bearing traversals.

Each search invocation starts with a fresh record.

10.2 Watching the guesser search

Fix root $q = 1$, belief $b_\ell(\text{O}) = 4/5$, $b_\ell(\text{B}) = 1/5$, terminal settlement, and one-step exploration. Try unvisited actions in order $g_{\text{O}}, g_{\text{B}}$, then choose the highest current mean, breaking ties toward g_{O} . The following is one possible sample sequence under b_ℓ . A cell records $(N, W, \widehat{Q})$ after the attempt.

Attempt	Type	Choice	Settlement	g_{O} statistics	g_{B} statistics
1	O	g_{O}	+1, terminal	(1, 1, 1)	(0, 0, -)
2	O	g_{B}	-1, terminal	(1, 1, 1)	(1, -1, -1)
3	B	g_{O}	-1, terminal	(2, 0, 0)	(1, -1, -1)
4	O	g_{O}	+1, terminal	(3, 1, 1/3)	(1, -1, -1)
5	O	g_{O}	+1, terminal	(4, 2, 1/2)	(1, -1, -1)
6	B	g_{O}	Execution failure	(4, 2, 1/2)	(1, -1, -1)

After attempt 2, $1 > -1$, so the selector chooses orange. After attempt 3 its mean falls to zero, still above blue's -1 . Six attempts have consumed the budget, and five have supplied values. The highest-mean final rule chooses g_{O} . Its backed mean $1/2$ differs from the exact target $3/5$ because this short realized sample contains only four orange-action returns.

The sixth attempt produces a typed failure and leaves both value counts unchanged. In the toy mathematics, we can calculate the unexecuted continuation: guessing orange under **B** would pay -1 . Had execution completed, orange's statistics would become $(5, 1, 1/5)$. This comparison explains the missing contribution; the implementation records the observed failure, without substituting that counterfactual payoff. A single trace illustrates arithmetic. The systematic selective-failure model in [section 12](#) explains bias across repetitions.

Connecting decisions in longer games

The guesser's choice ends the signal game. In a longer game, an action can lead to another decision node, and different hypotheses may produce different successors. We therefore need to describe how these nodes connect.

Definition 10.2 (Search connectivity). *A successor node n' is possible from (n, a) if there exists a valid history h with $K(h) = n$ such that, writing $h' = h \parallel (a, \text{next}(\text{last}(h), a))$, the successor is nonterminal and $K(h') = n'$.*

Each edge has a history that produces it, but a path through the graph may combine edges from different histories. Simulation follows the successor of its current hypothesis, which keeps the path consistent with one possible game.

Example 10.3 (Why the quotient need not be a tree). Consider two initial states with a hidden order of play. Each player makes one private choice; the other receives no observation of its content. In one initial state player 0 acts first; in the other player 1 acts first. Each player receives the same first-decision request when their turn arrives. The first-decision nodes can have an edge $n_0 \rightarrow n_1$ in the first case and $n_1 \rightarrow n_0$ in the second. The quotient graph has a cycle, although each actual history ends after two choices. Traversing that apparent cycle would splice incompatible order hypotheses. Own-choice recall prevents an actual player's second choice from being confused with their first.

This example concerns the graph across the allowed history family; a particular root's support can exclude some edges. Unfolding valid histories gives a tree of occurrences. Sharing information-state keys gives a graph. Neither data structure removes the need to retain a consistent simulated world.

10.3 How a node acquires a population

The node key specifies an information state, not a distribution over its compatible histories. A root belief, earlier selection policies, observed continuations, and sampling decisions determine the distribution of histories that reach a node during a particular search.

If ν is a declared distribution over histories in node n , its successor distribution after fixing a is obtained by pushing ν through advancement and K , with terminal outcomes represented separately. Uniformly choosing a successor node is not implied by the connectivity relation. Distinct nodes can aggregate different amounts of probability.

11 A complete search iteration

A search fixes its root belief and configuration and uses root-perspective payoffs. Algorithm 1 makes B attempts, calling Algorithm 2 to explore at most D actions before settlement. Fallible operations return $\text{OK}(x)$ or $\text{FAILURE}(\text{reason})$.

Algorithm 1 Bounded search with short-circuit failure handling

Require: Root (q, i_q) , belief b_q , configuration ξ , finite bounds B, D

- 1: Initialize search record Z ; on failure, return declared legal fallback
 - 2: **for** $j = 1, \dots, B$ **do** $\triangleright$ Each entry consumes one attempt, including failures
 - 3: $\eta \leftarrow \text{ATTEMPT}(q, i_q, b_q, Z, \xi, D)$
 - 4: **if** $\eta = \text{OK}(L, v, \kappa)$ **then**
 - 5: Back up v along L ; retain settlement origin κ
 - 6: **else**
 - 7: Record the failure reason; perform no numeric backup
 - 8: **end if**
 - 9: **end for**
 - 10: **return** Root distribution $\rho(\cdot \mid i_q, Z)$, with legal fallback
-

Inside `ATTEMPT`, the notation $x \leftarrow \text{TRY}(f)$ evaluates f , extracts x only from $\text{OK}(x)$, and otherwise *immediately returns* that failure from the attempt. This short-circuit rule applies before any later line runs. All operations, including initialization, sampling, final choice, and settlement, have finite work bounds and typed failure or a declared fallback.

Algorithm 2 One search attempt

```

1: function ATTEMPT( $q, i_q, b_q, Z, \xi, D$ )
2:    $h \leftarrow \text{TRY}(\text{Sample}(b_q)); L \leftarrow []$ 
3:   for  $d = 0, \dots, D$  do
4:     if  $\text{last}(h) \in F$  then
5:       return  $\text{OK}(L, u_q(\text{last}(h)), \text{TERMINAL})$ 
6:     end if
7:     if  $d = D$  then
8:        $(v, \kappa) \leftarrow \text{TRY}(\text{Settle}(h, q, \xi))$ 
9:       return  $\text{OK}(L, v, \kappa)$ 
10:    end if
11:     $n \leftarrow \text{TRY}(\text{ProjectNode}(h))$ 
12:     $M \leftarrow \text{TRY}(\text{Propose}(n, Z, \xi))$ 
13:     $(a, \text{new}) \leftarrow \text{TRY}(\text{Select}(n, M, Z, \xi))$ 
14:     $h' \leftarrow \text{TRY}(\text{Advance}(h, a))$ 
15:    Append  $(n, a)$  to  $L$ ; set  $h \leftarrow h'$ 
16:    if  $\text{new}$  then
17:       $\text{TRY}(\text{RegisterSuccessor}(n, a, h, Z))$ 
18:       $(v, \kappa) \leftarrow \text{TRY}(\text{Settle}(h, q, \xi))$ 
19:      return  $\text{OK}(L, v, \kappa)$ 
20:    end if
21:  end for
22: end function

```

`ProjectNode` constructs $K(h)$ in the supported representation; `Propose` and `Select` use the node's player information and the search record. `Advance` and `successor projection` operate on the trusted hypothesis. `Settle` checks terminal payoff first, then runs the declared evaluator or bounded continuation; failure propagates through the same `TRY` rule. Structural changes made before a failed attempt may remain in Z as recorded exploration, while value counts increase only at backup. A cutoff at depth zero can settle the root without evaluating any root action, so the final choice rule also handles an empty value-bearing path.

11.1 Selection from the current search record

At a node $n = (r, i_r)$, a computational selector returns a distribution over represented actions:

$$\alpha_r(\cdot \mid i_r, Z, \xi) \in \Delta(A_{\text{rep}}(n)).$$

Holding search record Z and configuration ξ fixed, it is a function of the node, not of inaccessible details of the current hypothesis. Hypothesis-dependent menu filtering or choice features can violate this fixed-record requirement even when the node keys themselves are safe.

An illustrative UCB-style score for visited actions is

$$\epsilon_q(n)\widehat{Q}(n,a) + c\sqrt{\frac{\log(1+N(n))}{N(n,a)}}, \quad N(n) = \sum_a N(n,a), \quad (16)$$

where $c \geq 0$ and $\epsilon_q(n)$ is $+1$ for a root-player node and -1 for an opponent node. The first term favors larger observed returns at root-player nodes and smaller ones at opponent nodes. For $c > 0$, the second gives less-sampled actions a larger exploration bonus, encouraging the search to revisit alternatives to an early favorite. Increasing c places more weight on exploration.

Unvisited actions need an initial exploration rule, and equal scores need a tie-break. This score illustrates the bandit-guided allocation used in UCT [2]; section 16 discusses the conditions of its analysis.

As Z evolves, the selection distribution changes with the accumulated statistics. At opponent nodes, the minimizing sign defines the search-conditioned response described in section 11.2. Prediction under a fixed behavioral opponent instead uses its policy. The model used to interpret past opponent choices may differ from the rules used to choose future actions during search and rollout.

11.2 Which opponent is being simulated?

A behavioral opponent policy $\mu_r(\cdot | i_r)$ gives the same action distribution at equal opponent information. The selector instead conditions on the search record as well: $\alpha_r(\cdot | i_r, Z, \xi)$. Across root searches, Z can carry private information that changes the response at the same opponent node.

Proposition 11.1 (Local selector invariance permits an opponent information advantage). *Sharing opponent information-state keys and obeying fixed-record selector invariance need not produce one behavioral opponent policy across root contexts.*

Counterexample. Run two sender-root searches in the signal game: the sender knows type **O** in one and type **B** in the other. In each search, consider sending ℓ . The guesser then faces the same information state i_ℓ . Each root search evaluates both guesses against its known type and stores their sender payoffs in $Z_{\mathbf{O}}$ or $Z_{\mathbf{B}}$. Minimizing sender payoff chooses $g_{\mathbf{O}}$ under $Z_{\mathbf{O}}$ and $g_{\mathbf{B}}$ under $Z_{\mathbf{B}}$. The selector uses only (i_ℓ, Z) , satisfying fixed-record invariance. A single guesser policy $\mu_1(\cdot | i_\ell)$ would have to give both guesses probability one, which is impossible. $\square$

Two information boundaries, two useful questions

Root perspective safety asks whether the final choice depends only on the root's legitimate information. Opponent realizability asks whether the modeled opponent's choices agree across all contexts with the same opponent information. The counterexample satisfies the first condition while failing the second.

For prediction under a specified opponent policy, simulation can use that policy directly. An optimization procedure seeking a consistent strategic opponent must constrain the induced kernels across the relevant information sets and root contexts. Local maximization or minimization of search statistics alone supplies a search-conditioned response. Cowling, Powley, and Whitehouse discuss the closely related private-card information advantage when the searching player's cards remain fixed across determinizations [3, Section X, pp. 140–141].

11.3 Expansion and action coverage

Write $A_{\text{rep}}(n) \subseteq A(n)$ for represented actions. Expansion may introduce all actions or grow this set progressively. A proposal rule is computational machinery; an omitted action remains legal. The actual simulation must bind a proposed semantic action to the corresponding action in its current hypothesis without changing its meaning.

This separates four implementation questions: is an action legal, was it proposed, was it admitted to the configured search, and did its attempted execution succeed? Their sets can differ without changing the theoretical legal set. When reporting a result, identify which legal actions the search considered.

11.4 Backup and root choice

For each occurrence (n, a) on the exploration path receiving settlement v ,

$$N(n, a) \leftarrow N(n, a) + 1, \quad W(n, a) \leftarrow W(n, a) + v.$$

The same root-perspective scalar is stored at every node. Opponent selection applies its sign when reading that scalar, as in [equation \(16\)](#). An acting-player storage convention is also possible if storage and selection use matching conversions.

Rollout choices need not create graph visits; their role is to supply the settlement for the recorded exploration path. Any scheme that also updates rollout nodes is an additional algorithm choice.

After budget B , let Z_B be the random search record. A root choice rule returns $\rho(\cdot \mid i_q, Z_B) \in \Delta(A_q(i_q))$. Examples include highest mean, highest visit count, and a declared distribution based on statistics. Ties, unvisited actions, and zero successful iterations require explicit handling. A legal fallback makes the procedure a total policy; without one, a procedure that can fail is only a partial policy implementation.

The induced search policy is

$$\pi_q^{\text{search}}(a \mid i_q) = \mathbb{E}[\rho(a \mid i_q, Z_B)], \quad (17)$$

where expectation includes sampling, exploration, and any final choice randomness. All persistent configuration and the budget convention are fixed when reading this expression. [Proposition 13.3](#) supplies sufficient conditions for this expression to depend only on i_q .

11.5 Evaluating a cutoff at the opponent's turn

Now start a different search at the sender's decision, with root $q = 0$ and known type $\textcircled{O}$. Consider the branch sending ℓ and stop at the next state, where the guesser acts. Fix the continuation model: the guesser always chooses $g_{\textcircled{O}}$. Define the sender's information-state evaluator on these waiting information states by its expected terminal payoff under that continuation:

$$e_0(I_0(h_{\textcircled{O}, \ell})) = -1, \quad e_0(I_0(h_{\textcircled{B}, \ell})) = +1.$$

The sender's own history identifies its type and sent signal. Any compatible history on this domain has the same terminal payoff under this policy, so every belief supported there yields the displayed value. At the $\textcircled{O}$ cutoff, `Settle` projects $I_0(h_{\textcircled{O}, \ell})$, calls e_0 , and backs -1 to the sender's ℓ branch with origin `ROOTINFORMATIONEVALUATOR`. The value is exact for this continuation, while its origin records that simulation stopped before the guesser chose.

The guesser's evaluator at i_ℓ , using b_ℓ and the same always-orange policy, would return $e_1(i_\ell) = 3/5$. Negating it would give $-3/5$, which differs from the sender's -1 because the guesser is uncertain about type. Calling the root-information evaluator preserves the root's subjective target even when the opponent is the next actor. A different settlement design may use a sampled-world scorer or a common-model rollout, with its own target.

12 Connecting search averages to a target

We can now relate the target, the evaluator, and the search average.

Object	Meaning
$Q_q(b_q, a; \pi)$	A specified terminal-payoff expectation under a belief and continuation profile.
$e_q(i_q)$	A scoring function on player information, with a separately specified target.
$\bar{Q}(n, a)$	The average of the values actually backed to a node-action pair.

Sampling, continuation, settlement, and the population reaching a node determine when a backed mean estimates a specified value target. The following cases show how to establish that connection and how it can change.

12.1 A fixed-action reference experiment

Suppose an experiment repeatedly samples $h \sim b_q$, fixes the same root action a , then follows the same continuation profile π to terminal payoff. Assume independent repetitions, almost-sure termination, and a fixed, deterministic number $m \geq 1$ of repetitions. Each return has expectation $Q_q(b_q, a; \pi)$, and their mean is an unbiased estimate of that target. Since returns are bounded, the sample mean converges almost surely to that target as the number of independent repetitions grows.

This fixed-action evaluation gives a useful reference experiment. In adaptive search, node visitation selects a population, deeper choices evolve, and stopping or execution can select which returns appear in the final mean. Each mechanism can be analyzed against the reference target.

12.2 Visitation changes the population

Consider a variant of the signal game with different signal probabilities. Let a root belief assign hidden type **O** and **B** probability 1/2 each. Suppose a simulated opponent who knows its type chooses an observed branch ℓ with probabilities 0.9 and 0.1, respectively. Conditional on reaching that branch, the hidden-type proportions are 0.9 and 0.1.

If a later fixed choice pays the root +1 under **O** and -1 under **B**, then

$$\mathbb{E}[v] = 0 \quad \text{under the root type prior}, \quad \mathbb{E}[v \mid \ell] = 0.9 - 0.1 = 0.8.$$

The second average describes the branch-conditioned population under that behavior model. Labeling it with its conditioning event explains why it differs from the prior expectation.

There is an equally important invalid version of the example. If one node represents the *same acting-player information state* for both types, and its selection rule directly uses hidden type to choose ℓ at those rates, the rule uses a distinction absent from the actor's information. Here the branch proportions arise from an information leak in selection.

12.3 Adaptive continuation, stopping, and execution

Even with independent returns and a fixed target, a data-dependent stopping rule can change the expected final mean. Let each return independently be $+1$ or -1 , equally likely. Stop after one sample when the first is $+1$; otherwise take a second sample. The possible means are 1 , 0 , and -1 , with probabilities $1/2$, $1/4$, and $1/4$. Thus

$$\mathbb{E}[\bar{v}_{\text{stop}}] = \frac{1}{2}(1) + \frac{1}{4}(0) + \frac{1}{4}(-1) = \frac{1}{4},$$

although every individual return has expectation zero. This effect comes from selecting when to report the average.

If continuation policy $\pi^{(j)}$ changes across successful iterations, the j th return can target a different conditional value. The pooled mean then combines a sequence of continuation targets. A convergence analysis must account for how that process evolves.

Similarly, suppose half the hypotheses would return $+1$ and half -1 , but all negative-return hypotheses fail before producing a value. Successful backups average $+1$. No failure was numerically scored as a win, yet the surviving population is biased for the intended all-hypothesis target.

12.4 Budgets, reuse, and uncertainty

A wall-clock budget can produce a different sample allocation from a fixed simulation budget, especially when runtime depends on the hypothesis. Report the actual budget and cutoff mechanisms so that work counts can be related to the completed trajectories they produced.

Reusing statistics under a new root belief requires more than matching keys. For example, returns collected under one distribution of remembered opponent information cannot simply be relabeled as samples from a different one. Importance weighting would require adequate support, known relevant sampling laws, and stable target semantics; this paper does not prescribe such a repair. The default formal search therefore starts with search-local statistics.

Confidence intervals also need a sampling model. Adaptive visits and selected successes are not automatically eligible for independent-sample formulas. The arithmetic establishes the empirical mean; the sampling model determines which uncertainty statement accompanies it.

Part IV

Representations and implementation

13 Representation contracts

The computer represents these mathematical objects with data structures. An engine object can encode a game state; a snapshot, event ledger, and knowledge summary can encode player information. An engine world together with both player records can encode the full epistemic tuple.

13.1 Perspective safety as factorization

Suppose an implementation constructs a policy-facing representation $\Phi_q : \mathcal{H} \rightarrow \mathcal{R}_q$. Perspective safety requires that it factor through the player information state:

$$\Phi_q(h) = r_q(I_q(h)) \quad \text{for some } r_q. \quad (18)$$

Equivalently, indistinguishable histories produce equal policy-facing representations. If representation construction randomizes, the corresponding requirement is equality of output distributions conditional on equal information states.

A representation can add computed consequences of observations without breaking this condition. But labels must still distinguish an entailed fact from a model-dependent belief. Randomization, hashes, identifiers, action order, cached annotations, and diagnostic fields are all potential channels and belong to the same policy-facing boundary.

13.2 The representation map

An implementation encodes information through

$$r_q : \mathcal{I}_q \rightarrow \mathcal{R}_q$$

and applies $\hat{e}_q : \mathcal{R}_q \rightarrow \mathbb{R}$. Its induced theoretical function is $\hat{e}_q \circ r_q$. Hence

$$r_q(i) = r_q(i') \implies \hat{e}_q(r_q(i)) = \hat{e}_q(r_q(i')).$$

No more powerful fitting algorithm can recover a distinction that the input representation has erased. A sufficient representation for one particular target can be incomplete for other tasks or for full information history.

Example 13.1 (Safe but incomplete). Two histories show the same current public board and hand sizes. One includes an earlier reveal of Lightning Bolt; the other includes an earlier reveal of Mountain. A representation retaining only the current board and counts can be perspective-safe but merges these information states. A scorer on that representation must give them the same score.

13.3 Completeness and task sufficiency

A representation is complete for theoretical player history when r_q is injective on realizable information states:

$$r_q(i) = r_q(i') \implies i = i'.$$

Combined with perspective safety, equality of representations is exactly equality of information states. Completeness concerns preservation of distinctions, not a requirement to store a literal uncompressed transcript. An exact reversible encoding qualifies.

Decision sufficiency is weaker: there exists a decoder $\widehat{D}_q$ with $\widehat{D}_q(r_q(i)) = D_q(i)$. Target sufficiency is also weaker: for a particular target f , there exists $\widehat{f}$ with $\widehat{f} \circ r_q = f$. These requirements allow a representation to omit parts of the history while preserving everything needed for a particular decision or scoring task.

Example 13.2 (Omitting an irrelevant observation). Extend the signal game by disclosing a decorative token $d \in \{\circ, \square\}$ at setup. Its distribution is independent of type, and the fixed sender model ignores it. At a guesser decision, let $r_1(i)$ retain only the observed signal, decision status, and legal menu. Histories with different d then have the same encoding, so r_1 is noninjective. Yet the menu is decoded exactly, and the posterior over type depends only on the signal.

For the always-orange continuation, the target $f(i)$ is $3/5$ after ℓ and $-3/5$ after r . Defining $\widehat{f}$ by these two values gives $f = \widehat{f} \circ r_1$: the compression is decision-sufficient and target-sufficient. Retaining the token would add no predictive information for this task. A sender model that responds to the token would change that conclusion, because signal likelihoods could then depend on the omitted detail.

13.4 Execution fidelity

Executing an action in the implementation should produce the encoded version of the state prescribed by the game model. Let $c : S \rightarrow \mathcal{X}$ encode a game state, and let $\text{bind}(s, a)$ give the engine's executable representation of action a in state s . For supported inputs, we require

$$\widehat{\text{next}}(c(s), \text{bind}(s, a)) = c(\text{next}(s, a)).$$

A refusal is an explicit failure, not an arbitrary alternative successor. The engine may need world-specific identifiers to execute an action. The policy-facing action representation must still respect the player's information boundary.

If the engine representation is richer than $c(s)$, the relation is better written using an abstraction map from engine objects to theoretical states. The requirement is still that represented legal execution commute with the theoretical transition. Extra engine bookkeeping does not establish extra player knowledge.

13.5 Incremental history and knowledge summaries

The stored player record can be updated as observations arrive. On reachable inputs, updating that record with new observations w should give the same result as encoding the full extended history $i \parallel w$:

$$\widehat{U}_q(r_q(i), w) = r_q(i \parallel w).$$

If the representation omits details required by later updates, it may fail this condition even when it is sufficient to score the current position.

A remembered-knowledge ledger efficiently summarizes consequences of player history. For a known-card draw, the update reads the previously established card identity, combines it with the new draw disclosure, and records the new location. An explicit update order preserves this derivation. Labels for facts and model-dependent inferences retain the distinction developed in [section 4](#).

13.6 Simulation and search-state isolation

Forking a sampled world must preserve both players' valid pasts while allowing independent future updates. Shared mutable histories, belief weights, or search statistics can contaminate otherwise valid branches. A world snapshot alone does not reconstruct either player's remembered experience.

An implementation key $\hat{K}$ realizes the chosen theoretical identity K when its equality classes match those of K . A coarser key pools additional information states. The sufficient-compression example shows why such pooling can be useful for a particular task; its adequacy depends on which decisions and continuation targets the abstraction preserves.

A finite digest alone cannot establish injectivity over an unbounded history space. An implementation that uses digests for keys therefore needs a way to handle collisions and determine semantic equality.

13.7 Composing a perspective-safe root policy

The root receives legitimate information and constructs its own hypothetical worlds. Those worlds can contain hidden details because they were generated by the root's model. A complete safety argument follows their provenance through the computation, including stopping and fallback behavior.

Proposition 13.3 (End-to-end root perspective safety). *Consider a search that terminates with a legal root action distribution. Assume the following conditions on its supported domain.*

1. *The root input is $i_q = I_q(h_{\text{actual}})$. Configuration, persistent inputs, belief construction, and initialization are fixed or factor through i_q .*
2. *All randomness and other external computational inputs have the same joint law at equal i_q . In particular, fresh random bits are independent of the actual hidden state conditional on the permitted inputs.*
3. *Every later operation—sampling, proposal, selection, advancement, settlement, backup, failure handling, stopping, and final choice—uses only these inputs and the hypotheses and records generated from them. Trusted simulation may inspect a generated hypothesis in full.*
4. *The bounds and fallback make the computation finite and total. Any external timing used to stop computation satisfies the identical-law condition above, including its dependence on the generated computation.*

Then two actual histories with the same I_q induce the same root action distribution, namely a behavioral policy $\pi_q^{\text{search}}(\cdot \mid i_q)$.

Proof. Couple two runs at equal root information using the same permitted inputs and randomness. Their initial beliefs and records agree. Inductively, each operation receives equal arguments, so its sampled hypothesis, successful output or failure, and updated record agree.

The same argument applies to stopping and fallback. Finite termination gives equal final action distributions in the coupling; averaging over its common input law proves the claim. $\square$

This proposition permits substantial computation on sampled hidden state. Its decisive restriction is that the actual hidden state contributes no extra input beyond i_q . Local selector invariance is useful for the chosen search model, but root safety follows from this broader provenance condition. Opponent realizability still asks the cross-context question in [proposition 11.1](#).

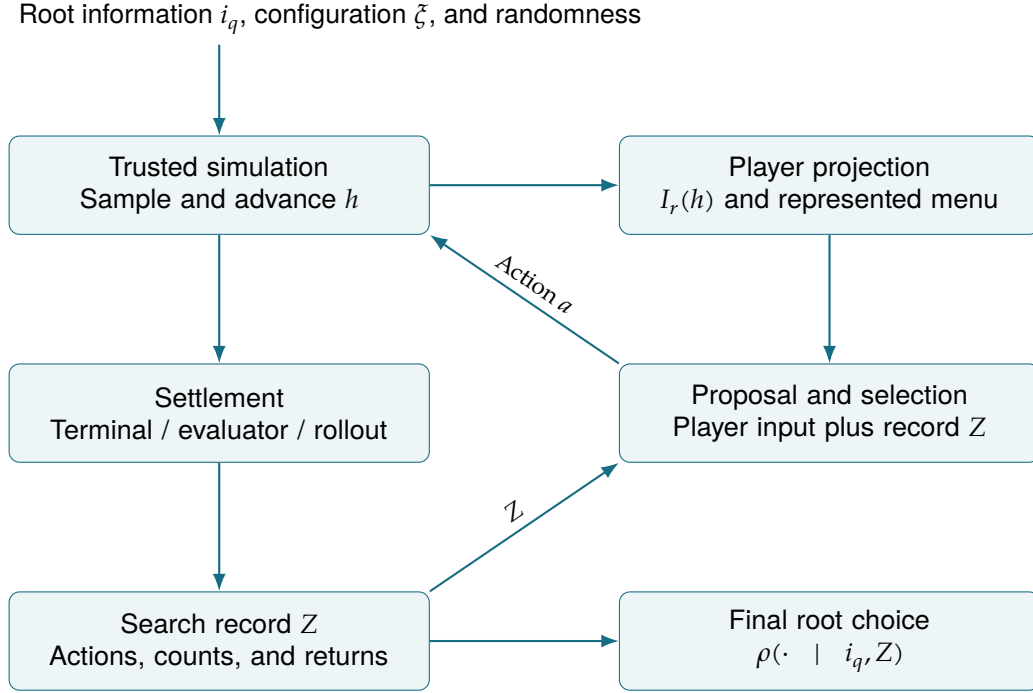


Figure 3: Information flow in a search invocation. Selection reads projected player information and the evolving record, then returns an action to advancement. Full hypotheses stay within simulation and settlement. All paths begin with permitted root inputs.

14 Symmetry between players

Exchanging players 0 and 1 carries ownership, control, private contents, and the role of playing first with the exchange. This changes the labels while preserving the strategic situation, including any advantage of playing first. For the signal game, the initial family also includes the states obtained by exchanging the sender's and guesser's player labels.

Let σ act on players, states, actions, and observations, with $\sigma(q) = 1 - q$ and σ^2 the identity on each domain. On observation words and histories it acts componentwise, preserving order. Require

$$\sigma(S_{\text{init}}) = S_{\text{init}}, \quad \sigma(F) = F, \quad (19)$$

$$p(\sigma(s)) = 1 - p(s), \quad A(\sigma(s)) = \sigma(A(s)), \quad (20)$$

$$\text{next}(\sigma(s), \sigma(a)) = \sigma(\text{next}(s, a)). \quad (21)$$

The nonterminal expressions apply only where they are defined. For observations,

$$O_{1-q}^0(\sigma(s_0)) = \sigma(O_q^0(s_0)), \quad (22)$$

$$O_{1-q}(\sigma(s), \sigma(a)) = \sigma(O_q(s, a)). \quad (23)$$

Proposition 14.1 (Equivariance of information and compatibility). *For every valid history h ,*

$$I_{1-q}(\sigma(h)) = \sigma(I_q(h)), \quad C_{1-q}(\sigma(i_q)) = \sigma(C_q(i_q)).$$

The corresponding game-state and epistemic-state sets transform in the same way, where $\sigma(s, i_0, i_1) = (\sigma(s), \sigma(i_1), \sigma(i_0))$.

Proof. The game equations make $\sigma(h)$ valid. Apply the observation equations to each term of [equation \(1\)](#). For compatibility, membership on either side follows by this identity; the reverse inclusion follows from $\sigma^2 = 1$. The statements about images follow by projecting the resulting sets. $\square$

Example 14.2 (First player versus player zero). An initial state may record that player 0 plays first and owns deck D . Its exchanged state records that player 1 plays first and owns the exchanged deck. Both states belong to the symmetric initial-state family. A convention that always writes the first player as 0 is a coordinate choice, not a reason to demand that an individually labeled initial state be fixed by σ .

Rule symmetry can relate instances with different decks, private information, and policies: the exchange transports each of these components to its new owner. Requiring equal components or an exchange-invariant prior adds restrictions on the particular instance or experiment.

14.1 Exchanging policies

Define the exchanged profile by

$$\pi_{1-q}^\sigma(\sigma(a) \mid \sigma(i_q)) = \pi_q(a \mid i_q).$$

The exchanged initial distribution is $\lambda^\sigma(\sigma(s)) = \lambda(s)$. Substituting into [equation \(8\)](#) gives

$$\mathbb{P}_{\lambda^\sigma, \pi^\sigma}([\sigma(h)]) = \mathbb{P}_{\lambda, \pi}([h]).$$

Requiring $\pi^\sigma = \pi$ is an additional assumption of symmetric behavior. The game symmetry itself only tells us how to exchange a possibly asymmetric profile.

14.2 Exchanging beliefs, values, and evaluators

Assume payoff respects player exchange: $u_{1-q}(\sigma(s)) = u_q(s)$. Transport a belief by σ and use the exchanged continuation profile. Pairing each continuation with its exchanged continuation preserves path probability and the appropriately labeled payoff. Under the domains and termination conditions of [section 7](#), this gives

$$\begin{aligned} V_{1-q}(\sigma_\# b_q, \pi^\sigma) &= V_q(b_q, \pi), \\ Q_{1-q}(\sigma_\# b_q, \sigma(a); \pi^\sigma) &= Q_q(b_q, a; \pi). \end{aligned}$$

A symmetric evaluator family likewise satisfies $e_{1-q}(\sigma(i_q)) = e_q(i_q)$. This compares exchanged perspectives in exchanged situations. Within one history, the players may have different subjective models, as the cutoff example in [section 11.5](#) illustrates.

15 Where these objects appear in MTGallium

The table points to the code representing each object and identifies what remains to be checked. Its links follow the public source layout:

main

Theory	Source representation	Mapped domain	What to check
Current visible information	SafeObservation-Projector	Viewer-specific Argentum observation to <code>PlayerObservationSnapshot</code>	Does every field, reference, and ordering factor through legitimate information?
Disclosures and remembered events	PerspectiveHistory ; PerspectiveEvent-Projector	Accepted choices, event projection, and visible transitions	Do supported events preserve order, own choices, and remembered knowledge?
Player information representation	ArgentumSearchWorld <code>informationState</code>	Snapshot, safe history, knowledge, and candidate choices	For which tasks is this representation sufficient? Which distinctions in I_q are preserved?
Evaluator input and settlement origin	SearchBoundary.kt	Information-state and sampled-world leaf-value interfaces	Does each call use the intended input domain, perspective, and target?

`PlayerObservationSnapshot` is a current snapshot, whereas the paper’s observation word accumulates disclosures through time. The source event pipeline combines new engine events with safe references and remembered knowledge. For example, a draw projection can include identities available to the drawer or justified by prior revelation and tracked library order. This is a concrete way to enrich a new event with previously derived knowledge.

The `argentum-board-v1` evaluator uses a sampled-world evaluation route. Visible V2, tactical scoring, and the linear and residual evaluators described in the [project glossary](#) use information-state inputs.

15.1 Application boundary of the deterministic model

Applying this framework to MTGallium requires checking the engine’s transitions, disclosures, and decision points. Drawing from a fixed hidden library order fits the deterministic model when that order is part of the state; fresh random shuffles require a chance-transition extension. The source map identifies code relevant to these checks, but does not establish that a complete game family satisfies the model. [Section 17](#) lists the model ingredients that such an extension must supply.

16 Connections to the literature

Extensive-form games supply histories, information sets, and behavioral strategies. Osborne and Rubinstein [1] provide a textbook treatment. Here the observation word is the *information state*; its fiber among acting histories is the information set. The word makes recall and chronological updates convenient to state.

Kocsis and Szepesvári [2, Section 2.4 and Theorem 6] analyze UCT in specified MDP and payoff-process settings. The selection expression in [equation \(16\)](#) illustrates the same bandit-guided allocation idea. Applying a convergence result here would require verifying its assumptions for the chosen hypothesis populations, adaptive continuations, and settlement rules.

Cowling, Powley, and Whitehouse [3, Sections II and IV.E] develop information-set Monte Carlo search and explain strategy fusion: combining choices across determinizations can grant a player distinctions it cannot observe. Their Section X addresses the searching player's private information being implicitly available to simulated opponents, directly motivating the cross-context question in [proposition 11.1](#). Their algorithms use particular tree and information-set conventions; the acting-player key $K(h)$ here is an explicit choice of statistical sharing for this specification.

17 Scope boundaries and a specification template

This model leaves engine chance transitions, truly simultaneous moves, nonterminating-play payoff conventions, intermediate rewards, discounting, learning updates, and equilibrium computation unspecified. Each extension needs its corresponding dynamics, information, and value conventions. Modeling simultaneous choices through hidden commitments, for example, requires an explicit disclosure and decision protocol.

To specify a particular search algorithm using this framework, fill in the following items:

1. **Game and information domain:** initial states, legal choices, advancement, disclosures, payoff, and supported representations.
2. **Belief model:** hypothesis object, prior or construction rule, historical behavior assumptions, endpoint, and approximation method.
3. **Target:** payoff perspective, future policy profile or strategic objective, and termination or cutoff meaning.
4. **Search structure:** node key, action identity, represented menu, and successor registration under consistent hypotheses.
5. **Computation:** proposal, selection, rollout, settlement, all budgets, failure handling, and backup conventions.
6. **Output:** root choice distribution, tie-breaking, fallback, and whether statistics are discarded or reused, and why reuse is valid.

The guesser's original question now has an answer under the chosen sender model: after ℓ , orange has probability $4/5$, so guessing orange has expected payoff $3/5$. A short search can favor that choice while reporting a different sample mean. We can explain the difference, track what a failed attempt leaves out, and recognize when a search record gives the modeled opponent an extra clue. These same links between observations, models, choices, and returns give a larger implementation its meaning.

A Notation reference

Symbol	Meaning
$P = \{0, 1\}$	Players; labels are distinct from the role of playing first.
S, S_{init}, F	Game states, allowed initial states, terminal states.
$p(s), A(s)$	Acting player and nonempty finite legal action set at a nonterminal state.
$\text{next}(s, a)$	Deterministic advancement through automatic consequences to the next game state.
$h, h_j, \mathcal{H}$	A valid finite game history, its prefix at depth j , and the set of all valid finite histories.
$\text{last}(h)$	The final game state of a history.
Ω_q, Ω_q^*	Player q 's observation alphabet and finite observation words.
O_q^0, O_q	Initial and transition observation-sequence functions.
$\varepsilon, \parallel$	Empty word and concatenation; history extension uses the analogous notation.
$I_q(h), \mathcal{I}_q$	Full player observation history and its realizable range; the theoretical information state.
R_q, D_q	Decoders of own-choice sequence and current decision status.
$\mathcal{I}_q^{\text{act}}$	Information states at which player q must choose.
$A_q(i_q)$	Legal actions decoded from an acting information state.
$C_q(i_q)$	Compatible game histories, without probabilities.
$E(h)$	Full epistemic state $(\text{last}(h), I_0(h), I_1(h))$.
$\mathcal{S}_q, \mathcal{E}_q$	Compatible current game states and compatible full epistemic states.
σ	Involutive player exchange, extended consistently to all labeled objects.
$\Delta(X)$	Probability distributions on finite or countable set X .
π_q, π	Behavioral policy and two-player policy profile.
$\lambda, [h]$	Initial-state distribution and event that play has prefix h .
$b_q, f_{\#}b_q$	Belief over compatible histories and its pushforward through f .
u_q	Terminal payoff to player q .
$G_q^\pi(h)$	Expected terminal payoff from a specified history under continuation profile π .
V_q, Q_q	Belief value and forced-next-action value for a specified continuation profile.
$e_q, \hat{e}_q$	Theoretical information-state scorer and scorer on a concrete representation.
r_q, Φ_q	Information-state encoding and its history-based constructor $\Phi_q = r_q \circ I_q$.
$K(h), n$	Acting-player information-state key and a decision node.
$A_{\text{rep}}(n)$	Legal actions currently represented at a search node.
$N, W, \widehat{Q}$	Successful value-bearing visit count, backed sum, and empirical mean.
(v, κ)	Settlement value and its declared origin.
Z, ξ, B, ρ	Search record, configuration, budget, and root choice rule.

B Compact contract sheet

- C1. Decision completeness.** *Model assumption; section 2.* Every nonterminal game state has one actor and a nonempty finite legal action set; terminal states have no outgoing choice.
- C2. Deterministic advancement.** *Model assumption; section 2.* Legal (s, a) determines exactly one successor after finite automatic processing. No additional choice is consumed.
- C3. Disclosure sufficiency.** *Model assumption; section 3.* $O_q(s, a)$ is determined by the chosen state and action; any needed current visibility context is represented.
- C4. Observation history.** *Definition; section 3.* I_q is concatenated disclosures, including initial information and correctly positioned own-choice observations.
- C5. Decision awareness.** *Model assumption; section 3.* Equal I_q values imply equal current decision status and, when acting, equal legal action sets.
- C6. Compatibility.** *Consequence; section 4.* The actual history lies in its compatibility class; joint epistemic components share one compatible witness.
- C7. Behavioral perspective.** *Definition; section 5.* Under a fixed policy, the action kernel depends only on the acting player's information state.
- C8. Symmetry.** *Model assumption and consequence; section 14.* Exchanging player identities commutes with advancement, observation, exchanged policies, and exchanged payoff perspectives.
- C9. Belief support.** *Definition and modeling choice; section 6.* Hypotheses lie within compatibility. Bayesian claims identify the past-generation law and a finite-history endpoint.
- C10. Value identity.** *Definition; section 7.* Expected values specify belief, payoff, continuation, and termination assumptions. A forced action is an intervention.
- C11. Consistent simulation.** *Implementation obligation; section 9.* The current hypothesis determines the successor. Genuine continuation policies use only their actor's information state. Search-conditioned selectors obey fixed-record invariance; interpreting them as one opponent policy also requires agreement across root contexts.
- C12. Settlement origins.** *Algorithm convention; section 9.* Terminal payoff, heuristic scoring, and bounded continuation retain distinct origins. Failure has no strategic value.
- C13. Population-aware backup.** *Algorithm convention and inference obligation; section 12.* Counts and means describe actual value-bearing traversals. Relating them to a target accounts for the effects of failure, selection, and changing continuations on the sampled returns.
- C14. Total output or explicit partiality.** *Implementation obligation; section 11.* Root selection stays legal and handles ties and absent values, or explicitly reports inability to choose.
- C15. Representation separation.** *Implementation obligation; section 13.* Safety, completeness, decision sufficiency, and execution fidelity are independently stated obligations.

C Exercises

Self-checks 1–12 revisit the main concepts; exercises 13–16 ask for a construction, proof, diagnosis, and calculation. Solutions begin in [section D](#).

1. reconstructibility

Suppose the engine draws the next card from a fixed hidden library. Does that violate deterministic advancement? What if it shuffles before drawing?

2. Empty observation sequences

A hidden opponent action changes no information available to player q , who is not asked to act before or after it. Why can $O_q(s, a) = \varepsilon$? What would change if a “hidden action occurred” token were appended instead?

3. equal board, unequal history

Two players’ current snapshots are equal in two histories, but one history contains a remembered reveal and the other does not. Can a complete information-state representation assign the same encoding?

4. a legal menu that leaks information

Two histories have the same proposed observation word, but action a is legal in only one. What part of the model must be revisited?

5. Learning from one’s own action

At information state i , a player selects a with probability $1/4$ in every compatible history. Does learning that they selected a change their belief over those histories before any further disclosure?

6. forced off-policy action

The continuation policy assigns action a probability zero. Is $Q_q(b_q, a; \pi)$ undefined?

7. zero-sum expectations

Player 0 assigns probability 0.8 to winning; player 1, using a different belief, assigns probability 0.7 to winning. With no draws, what are their subjective values, and is zero-sum payoff violated?

8. means and failures

An evaluator is exact, but the adapter fails on every hypothesis with negative payoff. Does omitting those failures from W and N ensure an unbiased estimate?

9. symmetry and action order

Two actions have equal root statistics. An implementation chooses whichever raw action identifier sorts first. Is player symmetry guaranteed?

10. full epistemic sufficiency

Two histories have the same (s, I_0, I_1) but different unobserved past events. Can their future payoff laws differ under the same information-state policies?

11. choosing the same node twice

Does sharing an information-state key permit a simulation to pick any outgoing successor of that graph node?

12. evaluator versus expected value

A learned scorer returns 0.6 at a nonterminal information state. List the additional information needed before interpreting it as an expected payoff.

13. Construct an observation encoding

For the signal game, construct typed initial and transition observation words for both players. Include enough information to recover own choices and current decision status. Give the guesser word after ℓ and explain why it is equal under both types.

14. Prove prefix-freeness

Prove that an acting compatibility class is prefix-free using only own-choice recoverability and decision consistency. Explain why the same argument need not apply to a waiting class.

15. Diagnose two selector inputs

Two hypotheses have equal guesser information i_ℓ . Selector A reads their hidden type directly to choose the matching guess. Selector B reads only (i_ℓ, Z) , but Z was built from a sender-root search that knows the type. Which local and cross-context conditions can each violate?

16. Calculate the missing return

Continue the trace in [section 10.2](#) with a seventh attempt that successfully guesses orange under type B. Compute both action records. Then compare the orange mean with a counterfactual execution in which attempt 6 also completed. Explain which record the actual algorithm may use.

D Worked solutions

1. reconstructibility

A fixed library order belongs to the game state, so drawing its next card can be deterministic even though a player does not know the identity. A stochastic shuffle is outside the present transition model. Hiding its random seed from the policy does not make the theoretical operation deterministic unless one explicitly changes what the state and model include. This paper does not make that extension.

2. Empty observation sequences

Empty output leaves player history unchanged. The token would distinguish histories according to occurrence or count of that hidden action. It is valid only if that occurrence really is disclosed. If the transition changes whether q must act, decision awareness requires a distinguishing observation, so the empty-output premise would no longer apply.

3. equal board, unequal history

No. Completeness requires injectivity on theoretical information states. A snapshot-only encoding may still be safe and decision-sufficient, but it is not complete for full observation history.

4. a legal menu that leaks information

Decision decoding cannot map the same word to two different legal sets. The information state must include the legitimately disclosed menu distinction, the action abstraction must be corrected, or the claimed observation model is invalid. Removing a from the menu only in sampled worlds does not preserve an exact shared information-state node.

5. Learning from one's own action

No: multiplying every prior mass by $1/4$ and normalizing returns the same belief. An opponent's action may have different likelihoods across its different private information states, so the corresponding argument does not generally apply to observed opponent behavior.

6. forced off-policy action

Not for that reason. The first action is fixed by definition; only later choices follow π . The action must be legal throughout the root compatibility class, and the resulting continuation must satisfy the payoff and termination assumptions. Ordinary conditioning on a zero-probability observed action would be a different operation.

7. zero-sum expectations

Their values are 0.6 and 0.4. There is no violation: the expectations use different probability distributions. Under a common distribution, the two payoff expectations would be negatives of each other.

8. means and failures

No. It avoids assigning a strategic value to failure, but conditions the empirical population on successful execution. A claimed all-hypothesis target still lacks the missing negative-return population.

9. symmetry and action order

No. If the ordering is not preserved under player exchange, the tie-break can change behavior under relabeling. A symmetric semantic order, or a symmetric distribution over ties, is needed for the stated exchange law. The same concern applies to proposal truncation.

10. full epistemic sufficiency

Not under the stated sufficiency assumptions: their next action law, successor state, and appended observations agree recursively. If the laws do differ, a fact omitted from the tuple affects transition, disclosure, payoff, or policy memory, so the model's sufficient-state or policy contract was false.

11. choosing the same node twice

No. Edges have existential witnesses that can be different histories. The sampled hypothesis must witness the chosen action's actual successor. The quotient graph represents statistical sharing, not an autonomous world simulator.

12. evaluator versus expected value

At least the payoff convention, target belief construction, continuation policies or strategic objective, supported input domain, and any output transformation. Even then it is an estimate whose validity needs evidence; its numerical range and input type do not establish accuracy.

13. Construct an observation encoding

Use initial tokens $\text{type}(\theta)$ and $\text{request}(\{\ell, r\})$ for the sender, and waiting for the guesser. After signal ℓ , append $\text{own}(\ell)$, waiting to the sender and $\text{signal}(\ell), \text{request}(\{g_O, g_B\})$ to the guesser. After the guess, append its own-action token to the guesser and a terminal token to both, together with declared payoff disclosure. The guesser word after ℓ is $[\text{waiting}, \text{signal}(\ell), \text{request}(\{g_O, g_B\})]$ under both types. Filtering own-action tokens recovers choices, and the latest request, waiting, or terminal token decodes status.

14. Prove prefix-freeness

If compatible h is a proper prefix of compatible h' , player q acts at the end of h and its chosen action occurs in h' . Thus $\text{own}_q(h')$ strictly extends $\text{own}_q(h)$. Equal information states would decode equal own-choice sequences, a contradiction. In a waiting class the intervening actions can all belong to the opponent and emit no observation to q , so neither the observation word nor its own-choice sequence need grow.

15. Diagnose two selector inputs

A violates fixed-record invariance: changing the current hypothesis with i_ℓ , Z fixed changes its action. B can satisfy that local rule while violating global opponent realizability, as in [proposition 11.1](#). Both computations can still be root-perspective-safe if the hidden hypothesis and record were generated entirely from legitimate root inputs; using the actual referee type beyond those inputs would violate [proposition 13.3](#).

16. Calculate the missing return

After the failed sixth attempt, orange has $(N, W) = (4, 2)$. Attempt 7 adds -1 , producing $(5, 1, 1/5)$; blue remains $(1, -1, -1)$. If attempt 6 had also completed with -1 , orange would instead have $(6, 0, 0)$. The actual backup uses the former record and retains the sixth failure in attempt accounting. Counterfactual completion is an explanatory calculation, not an observed return.

References

- [1] Martin J. Osborne and Ariel Rubinstein. *A Course in Game Theory*. MIT Press, 1994. [Authors' book page](#); [authorized electronic text](#).
- [2] Levente Kocsis and Csaba Szepesvári. "Bandit based Monte-Carlo Planning." In *Machine Learning: ECML 2006*, LNCS 4212, pp. 282–293, 2006. [Author-hosted paper](#).
- [3] Peter I. Cowling, Edward J. Powley, and Daniel Whitehouse. "Information Set Monte Carlo Tree Search." *IEEE Transactions on Computational Intelligence and AI in Games*, 4(2):120–143, 2012. [doi:10.1109/TCIAIG.2012.2200894](#). [University repository copy](#).